

FPT UNIVERSITY — LAB211

ĐỀ TÀI THỰC HÀNH



E-Commerce Flash Sale Simulation

Hệ thống Mô phỏng Thương mại điện tử — Ngăn chặn Âm kho

BIG QUESTION:

"Khi nhiều luồng xử lý đơn hàng đồng thời trên cùng một file CSV kho hàng — race condition nào xảy ra, và kỹ thuật đồng bộ hóa nào ngăn được tình trạng âm kho mà không làm throughput giảm quá 30%?"

Mã môn học LAB211 — OOP with Java	Kiến trúc bắt buộc MVC (Model-View-Controller)
Cấp độ Java OOP Nâng cao — Semester 3	Dữ liệu File CSV — tổng ≥ 10.000 dòng
Thời gian thực hiện 10 tuần (project nhóm)	Sản phẩm nộp Code + Doc + Slide + AI Log

⚠ LƯU Ý QUAN TRỌNG

Đây là project nhóm (2-4 thành viên). Mỗi thành viên phải có AI Log riêng ghi lại toàn bộ tương tác với AI trong quá trình làm dự án — đây là cơ sở cho phần AI Reflection (30% tổng điểm).

1. CÂU HỎI NGHIÊN CỨU (BIG QUESTION)



RESEARCH QUESTION

"Khi nhiều luồng xử lý đơn hàng đồng thời trên cùng một file CSV kho hàng — race condition nào xảy ra, và kỹ thuật đồng bộ hóa nào ngăn được tình trạng âm kho mà không làm throughput (tốc độ xử lý) giảm quá 30%?"

1.1. Tại sao đây là vấn đề quan trọng?

Trong hệ thống Flash Sale thực tế (Shopee Flash Deal, Lazada Super Sale, Tiki Deal...), vấn đề âm kho là rủi ro nghiêm trọng nhất về tính toàn vẹn dữ liệu:

- **Âm kho (Negative Stock):** Sản phẩm chỉ còn 1 đơn vị nhưng được bán thành công cho 3-4 khách hàng khác nhau — hệ thống phải huỷ đơn sau, gây mất uy tín.
- **Root cause — Race Condition:** Ba bước "đọc tồn kho → kiểm tra đủ hàng → trừ kho" không phải atomic trên file CSV khi nhiều thread cùng truy cập.
- **Cửa sổ nguy hiểm (Race Window):** Thread-1 đọc stock=1, Thread-2 đọc stock=1, cả hai thấy đủ hàng, cả hai ghi stock=0, kết quả thực tế stock=-1.
- **Trade-off cốt lõi:** Khóa quá chặt → throughput giảm mạnh, khách hàng chờ lâu. Không khóa → âm kho, dữ liệu sai. Mục tiêu: an toàn mà throughput không giảm quá 30% so với baseline.

1.2. Research Question Lite — So sánh thực nghiệm



RESEARCH QUESTION (LITE) — chứng minh bằng số liệu

"Transaction isolation level nào ngăn được race condition âm kho trong Flash Sale — và đánh đổi gì về throughput?"

Nhóm phải implement và so sánh 4 cơ chế sau bằng Simulator Tool, chứng minh bằng bảng số liệu thực:

- ...

Kết quả: Biểu đồ throughput (đơn/giây) vs. tỉ lệ âm kho (%) vs. tỉ lệ retry. Cơ chế nào đạt mục tiêu: 0% âm kho VÀ throughput không giảm quá 30%?

2. TỔNG QUAN BÀI TOÁN

Nhóm xây dựng hệ thống phần mềm mô phỏng toàn bộ quy trình Flash Sale trên sàn thương mại điện tử: tổ chức sự kiện giảm giá, xử lý đơn hàng đồng thời từ nhiều khách hàng, và quản lý tồn kho trên file CSV. Vấn đề cốt lõi cần giải quyết là ngăn chặn tình trạng âm kho khi nhiều thread tranh nhau đặt hàng cùng một sản phẩm giới hạn.

2.1. Bối cảnh nghiệp vụ

- Flash Sale kéo dài 1-2 giờ — sản phẩm giảm giá 30-70%, số lượng giới hạn cực thấp
- Mỗi sản phẩm Flash Sale có `limitedQty` (giới hạn) và `soldQty` (đã bán) — bất biến: $soldQty \leq limitedQty$
- Mỗi khách hàng chỉ được đặt tối đa 2 đơn vị cùng sản phẩm trong 1 sự kiện
- Hệ thống xử lý đơn hàng theo hàng đợi FIFO — nhưng nhiều thread xử lý song song
- Toàn bộ dữ liệu kho hàng lưu trong file CSV — thao tác I/O là nguồn gốc race condition
- Khách hàng VIP/PREMIUM được ưu tiên — nhưng không được phép phá vỡ ràng buộc tồn kho

2.2. Các đối tượng dữ liệu chính

Entity (Model)	Thuộc tính chính	File CSV tương ứng
Product		products.csv
FlashSaleEvent		flash_events.csv
FlashSaleItem		flash_items.csv ← CRITICAL
Customer		customers.csv
Order		orders.csv
OrderDetail		order_details.csv
OrderTransaction		transactions.csv

🔴 **Trường `version`** trong `FlashSaleItem` (và `Product`) là số nguyên tăng dần mỗi lần `soldQty` được cập nhật. Đây là nền tảng của Optimistic Locking — thread chỉ được ghi khi `version` đọc ra = `version` hiện tại trong file. Nếu `version` đã thay đổi (thread khác đã ghi trước), phải retry hoặc từ chối đơn.

3. KIẾN TRÚC MVC BẮT BUỘC

⚠ **BẮT BUỘC:** Toàn bộ hệ thống phải tuân thủ MVC. Logic trừ kho, tính giá, validate giới hạn mua phải nằm trong Model/Repository — không được đặt trong Controller hay View.

3.1. Phân tầng trách nhiệm MVC

Lớp	Trách nhiệm	Được phép làm	KHÔNG được làm
Model	Dữ liệu + nghiệp vụ kho hàng	Đọc/ghi CSV, trừ kho, validate giới hạn mua, quản lý version, xử lý conflict	Hiển thị UI, nhận input
View	Hiển thị + nhận input	In menu, bảng sản phẩm, nhận input từ người dùng, gọi Controller	Đọc/ghi file, tính toán kho
Controller	Điều phối flow	Nhận request từ View, gọi Model, chọn cơ chế lock, trả kết quả về View	Trực tiếp đọc/ghi file, hiển thị UI

3.2. Lược đồ Class — Toàn bộ hệ thống

Mô tả dưới là ngôn ngữ tự nhiên. Nhóm phải vẽ thành UML Class Diagram đầy đủ (attributes, methods, visibility +/-#, multiplicity 1..*, 0..1) và nộp kèm báo cáo.

4. FLOWCHART CÁC LUỒNG CHÍNH

Nhóm phải vẽ các flowchart dưới đây bằng tool (draw.io, Lucidchart, PlantUML) và nộp kèm báo cáo. ASCII diagram dưới là bản phác thảo logic — flowchart thực tế cần có hình thoi (quyết định), hình chữ nhật (xử lý), và mũi tên có nhãn điều kiện.

4.1. Luồng Đặt Hàng Flash Sale (Order Placement Flow)

4.2. Luồng Ngăn Âm Kho — So sánh 4 Cơ chế (Race Condition Flow)

4.3. Luồng Simulator Tool (CountDownLatch)

4.4. Luồng Generate Data CSV (≥ 10.000 dòng)

5. HƯỚNG DẪN IMPLEMENTATION

5.1. Custom Exceptions bắt buộc

5.2. Simulator với CountdownLatch — Code mẫu

5.3. Đo Throughput và So sánh

6. LỘ TRÌNH THỰC HIỆN — 10 TUẦN

Tuần	Milestone	Công việc chi tiết	Deliverable
T1–T2	 Phân tích & Dữ liệu	Phân tích yêu cầu, vẽ Use Case Diagram. Thiết kế Class Diagram chi tiết. Thiết kế schema 6 file CSV (columns, types, FK). Viết DataGenerator.java sinh ≥10,000 dòng vào CSV. Kiểm tra parse/serialize round-trip.	Class diagram. CSV schema doc. File dữ liệu mẫu ≥10k dòng. DataGenerator chạy thành công.
T3	 Model Layer	Implement tất cả Entity class (Product, FlashSaleItem, Customer, Order, OrderDetail, OrderTransaction). Implement Enum (SaleStatus, OrderStatus, CustTier, LockMechanism...). Implement abstract BaseEntity + toCsvLine/fromCsvLine. Unit test parse CSV đúng.	Model classes + Enum. Test parse/serialize pass 100%.
T4	 Repository Layer	Implement CsvRepository<T> generic đầy đủ. Implement FlashSaleItemRepository (quan trọng nhất — có sellWithOptimisticLock và sellWithFileLock). Implement các repository còn lại. Test CRUD đọc/ghi file chính xác.	Tất cả Repository hoạt động. Đọc 10k dòng < 1 giây.
T5	 Controller Layer	Implement CustomerController (register, login). Implement FlashSaleController (start/end event, list items). Implement OrderController với NO_LOCK trước (baseline). Test đặt hàng đơn luồng thành công.	Booking single-thread thành công. Test case đặt hàng pass.
T6	 View Layer + MVC Wiring	Implement MainView, FlashSaleView (bảng sản phẩm), OrderView, ReportView. Kết nối View ↔ Controller ↔ Model đúng MVC. Không có bất kỳ logic kho hàng nào trong View hoặc Controller trực tiếp.	Demo booking flow end-to-end từ UI. Code review: xác nhận MVC đúng.
T7	 Synchronization Mechanisms	Implement FILE_LOCK (FileLock NIO với try-with-resources). Implement SYNCHRONIZED (synchronized method trong FlashSaleItemRepository). Implement OPTIMISTIC (version field + retry loop, MAX_RETRY=3). Test từng cơ chế với 2-5 threads thủ công, xác nhận soldQty không vượt limitedQty.	3 cơ chế hoạt động đúng. Test 5 threads: 0 âm kho với FILE_LOCK/SYNC/OPTIMISTIC.
T8	 Simulator Tool	Implement SimulatorController với CountdownLatch + ExecutorService. Implement SimulatorView: bảng so sánh ASCII với cột "vs Baseline" và "Mục tiêu". Chạy simulation 100–500 threads. Ghi kết quả vào transactions.csv. Đo throughput chính xác (TPS = success/elapsed).	Demo live simulator 500 threads × 4 mechanisms. Bảng so sánh in đúng.
T9	 Research & Báo cáo	Chạy full experiment: 1000 threads × 4 mechanisms × 3 lần lặp (lấy trung bình). Vẽ biểu đồ throughput vs. âm kho rate. Trả lời Research Question: cơ chế nào đạt 0% âm kho VÀ throughput không giảm quá 30%? Viết báo cáo Word + Slide ≥15 trang.	Báo cáo Word hoàn chỉnh. Slide. Biểu đồ so sánh 4 cơ chế.

T10	 AI Reflection & Nộp	Tổng hợp AI Log cá nhân từng thành viên. Viết AI Reflection: quá trình dùng AI, đánh giá chất lượng output, lỗi AI mắc, bài học rút ra. Review code toàn bộ, fix bug, polish UI. Đóng gói ZIP theo cấu trúc yêu cầu.	AI Log file. AI Reflection trong báo cáo (≥500 từ/người). ZIP nộp hoàn chỉnh.
-----	---	--	--

 **Checkpoint quan trọng:** Tuần 4 (FlashSaleItemRepository với 3 cơ chế lock) và Tuần 8 (Simulator chính xác) là 2 điểm kỹ thuật khó nhất. Nếu Tuần 4 không xong đúng hạn, toàn bộ phần sau sẽ bị ảnh hưởng domino.

RUBRIC CHẤM ĐIỂM

Tổng điểm: 100%. Không có điểm bonus. Điểm nhóm = điểm cơ sở. Điểm cá nhân = điểm nhóm × hệ số đóng góp (đánh giá qua AI Reflection và peer review).

Tiêu chí	Trọng số	Mô tả chi tiết	Bằng chứng đánh giá
1. Decomposition	20%	Phân tích đúng yêu cầu thành các lớp có trách nhiệm rõ ràng (SRP). Model / Repository / Controller / View không bị chồng lấn — đặc biệt: toàn bộ logic trừ kho và đồng bộ hóa nằm trong Repository, không rò ra Controller hay View. Xác định đúng 6 Entity và quan hệ giữa chúng.	Code review: grep tìm logic "soldQty" trong View/Controller → phải không có. Diagram Use Case đúng.
2. Abstraction	15%	UML Class Diagram đầy đủ (visibility, attributes, methods, multiplicity) + 4 Flowchart (Order, Race Condition, Simulator, DataGen). Hierarchy rõ: BaseEntity → Entity. Generalization: CsvRepository<T> áp dụng cho mọi entity. Enum thay string magic.	Nộp file .drawio/.png. Confirm CsvRepository<T> có thể dùng cho Product, Customer, Order, FlashSaleItem mà không sửa code.
3. Algorithm Design + Pattern Recognition	15%	CRUD đầy đủ cho Product, Customer, FlashSaleEvent, FlashSaleItem (thêm/sửa/xóa/tìm kiếm theo điều kiện). File I/O ổn định: try-with-resources, không để file bị corrupt. ≥5 custom exceptions đúng ngữ cảnh. DataGenerator sinh đúng ≥10,000 dòng CSV, parse round-trip không mất dữ liệu.	Demo tìm sản phẩm theo category + khoảng giá. Run DataGenerator → đếm dòng. Restart chương trình → data vẫn còn.
4. Algorithm Design (Simulator)	20%	Simulator chạy đúng: N threads đồng thời thực sự (CountDownLatch.countDown() trước khi threads race). Implement đúng ≥3 cơ chế đồng bộ, đo throughput (TPS) chính xác. Kết quả thực nghiệm hợp lệ: NO_LOCK có âm kho, các cơ chế kia không có. Bảng so sánh rõ ràng với cột "vs Baseline %" và "Mục tiêu".	Demo live: NO_LOCK với 500 threads → thấy âm kho. OPTIMISTIC → 0 âm kho. Số TPS khớp với bảng báo cáo.
5. AI Reflection	30%	Mỗi thành viên nộp AI Log cá nhân (log thực — raw conversations, không tô vẽ). Phân tích: AI đã hỗ trợ gì (tốt/kém), những output AI sai như thế nào, đã chỉnh sửa ra sao. Bài học: kỹ năng prompt, nhận biết hallucination, giới hạn AI với concurrent programming. Đánh giá: mức phụ thuộc AI có ảnh hưởng đến việc học không?	AI Log file có timestamp thực. Reflection ≥500 từ/thành viên. Có ví dụ cụ thể: [prompt] → [AI output] → [lỗi phát hiện] → [fix thực tế].
TỔNG CỘNG	100%	AI Reflection 30% — tiêu chí cá nhân quyết định điểm của từng thành viên, không phải điểm nhóm.	

7.1. Mức điểm AI Reflection (30%)

Mức	Điểm	Mô tả
Xuất sắc	27–30%	AI Log đầy đủ, phân tích sâu lỗi AI (đặc biệt về concurrent programming), chỉ ra cụ thể hallucination, thể hiện tư duy phản biện mạnh, rút ra bài học có thể áp dụng.
Tốt	21–26%	AI Log đầy đủ, có phân tích với ví dụ cụ thể, nhưng chưa đủ chiều sâu về giới hạn AI với concurrency.
Đạt	15–20%	AI Log có nhưng thiếu chi tiết. Reflection chung chung, ít ví dụ, chủ yếu mô tả lại bài làm.
Không đạt	0–14%	Không có AI Log, hoặc Reflection là mô tả lại bài làm mà không có đánh giá quá trình dùng AI.

7.2. Các lỗi bị trừ điểm

- **Logic trừ kho (soldQty) trong View hoặc Controller:** -5%/lần phát hiện
- **Simulator không dùng CountdownLatch (threads không đồng thời thực sự):** -8% (kết quả thực nghiệm không hợp lệ)
- **DataGenerator không đủ 10,000 dòng tổng:** -5%
- **NO_LOCK không có âm kho (simulator setup sai):** -5% (research question không thể trả lời)
- **AI Log không tồn tại hoặc rõ ràng giả mạo:** 0% phần AI Reflection cá nhân đó
- **Không compile được:** 0% toàn bài

8. YÊU CẦU SẢN PHẨM NỘP

Cấu trúc ZIP nộp bài:

```
// Java
NHOM_XX_LAB211_FlashSale.zip
├── src/
│   ├── model/
│   ├── repository/
│   ├── controller/
│   └── view/
├── data/
│   ├── products.csv
│   ├── customers.csv
│   ├── flash_events.csv
│   ├── flash_items.csv
│   ├── orders.csv
│   ├── order_details.csv
│   └── transactions.csv
├── docs/
│   ├── report.docx
│   ├── slide.pptx
│   ├── class_diagram.png
│   └── flowcharts/
├── DataGen/
│   ├── ai_logs/
│   │   ├── member1_ai_log.md
│   │   ├── member2_ai_log.md
│   │   └── ...
└── ...
```

← toàn bộ source code Java (cấu trúc MVC)

← Entity, Enum, BaseEntity

← CsvRepository<T>, các Repo cụ thể

← OrderController, SimulatorController...

← MainView, FlashSaleView, SimulatorView...

← file CSV đã generate

(≥ 5,000 dòng)

(≥ 2,000 dòng)

(≥ 500 dòng)

(≥ 2,500 dòng)

(≥ 2,500 dòng)

(kết quả simulation)

← báo cáo Word đầy đủ

← slide trình bày

← UML Class Diagram

← 4 flowchart (Order / RaceCondition / Simulator / ...)

← AI Log cá nhân từng thành viên

└─ README.md
chạy Simulator

← compile, chạy DataGenerator, chạy chương trình,

- Tên file ZIP: NHOM_XX_LAB211_FlashSale.zip (XX = số nhóm, ví dụ: NHOM_03_LAB211_FlashSale.zip)
- README phải có: lệnh compile, lệnh chạy DataGenerator, cách chạy Simulator và đọc kết quả
- Nộp qua FPT University Learning trước deadline được thông báo trên lớp
- Trễ 1 ngày: -10% | Trễ 2 ngày: -30% | Trễ ≥ 3 ngày: 0 điểm

Chúc các nhóm coding hiệu quả và khám phá thú vị! ⚡

"Concurrency bugs are not bugs you find — they are bugs that find you, at the worst possible moment."